

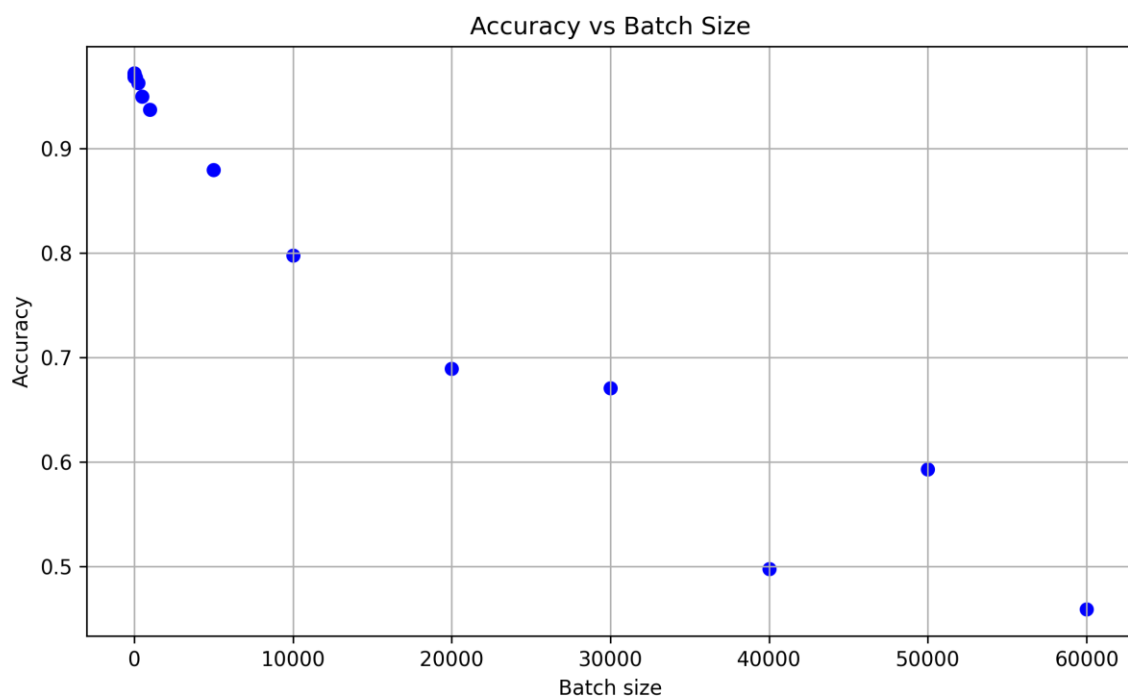
Sprawozdanie: Perceptron wielowarstwowy

Skład grupy: Łukasz Kiersnowski, Michał Lewczuk

Zad1 - Analiza wpływu wielkości batch-size na proces uczenia:

Przeanalizowaliśmy wpływ wielkości partii danych na dokładność (accuracy), wartość funkcji straty (loss) oraz czas uczenia (time) modelu dla 16 różnych wartości parametru *batch size*.

Batch size	Loss	Accuracy	Time (s)
1	0.1496	0.9685	419.16
2	0.1039	0.9720	217.55
5	0.1030	0.9711	87.12
10	0.0930	0.9696	45.77
50	0.0923	0.9702	11.01
100	0.1045	0.9677	6.49
250	0.1276	0.9628	3.84
500	0.1676	0.9497	2.99
1000	0.2175	0.9370	2.16
5000	0.4574	0.8796	1.69
10000	1.1080	0.7974	1.65
20000	1.8174	0.6892	1.62
30000	2.0191	0.6706	1.58
40000	1.9984	0.4973	1.66
50000	1.9339	0.5929	1.69
60000	2.1822	0.4589	1.87



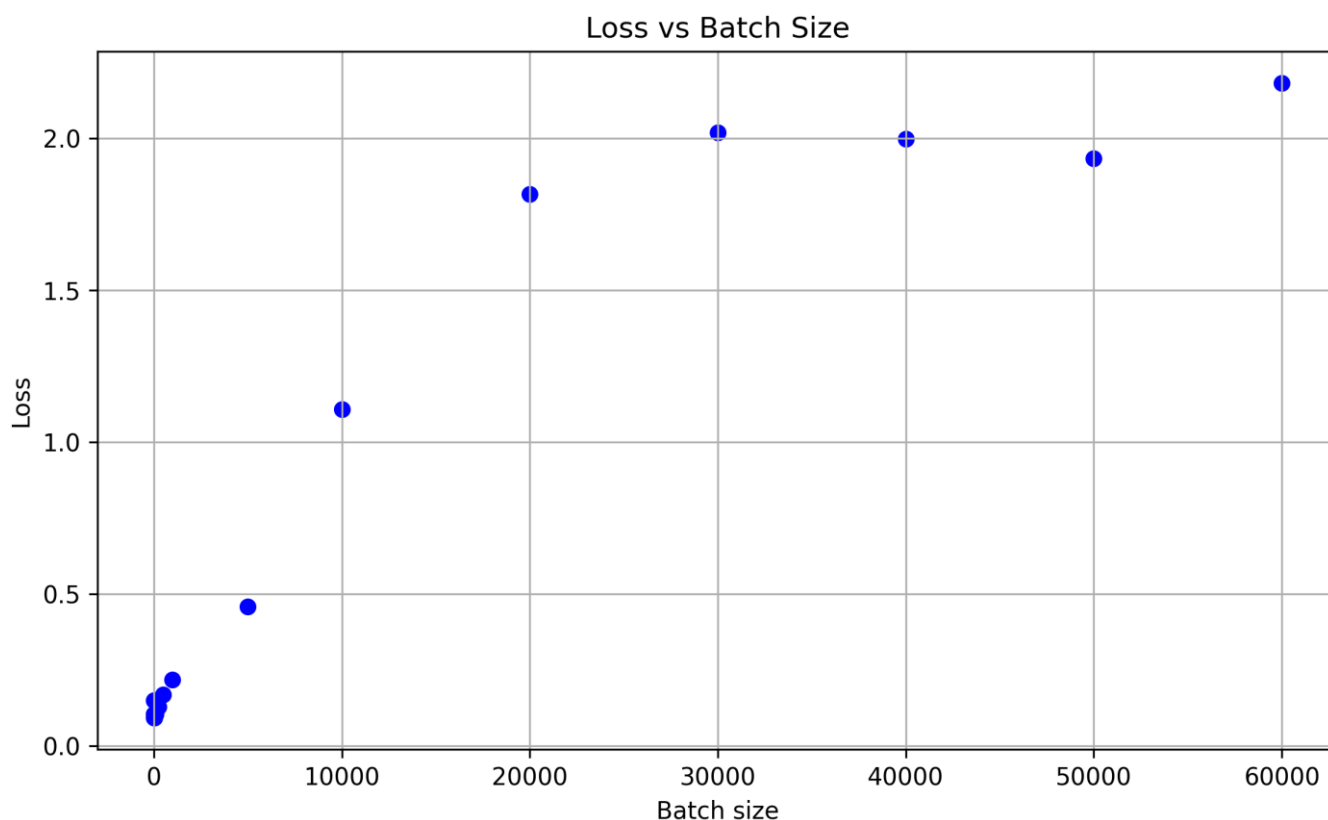
Wraz ze wzrostem wielkości partii danych dokładność modelu spadała, z wyjątkiem anomalii przy *batch size* = 50000. Najwyższą dokładność model osiągnął dla *batch size* = 2, uzyskując wartość 0.9720. Wysoki poziom dokładności — powyżej 0.96 — utrzymywał się dla wszystkich wartości od 1 do 250.

Uważamy, że spadek dokładności wynika z mniejszej liczby aktualizacji wag modelu przy większych partiach danych. Potwierdza to obserwacja, że dla *batch size* = 60000, po zwiększeniu liczby epok z 3 do 500, model osiągnął dokładność 0.9691 w czasie 124,72 s, czyli około 3,5 razy szybciej niż przy *batch size* = 1.

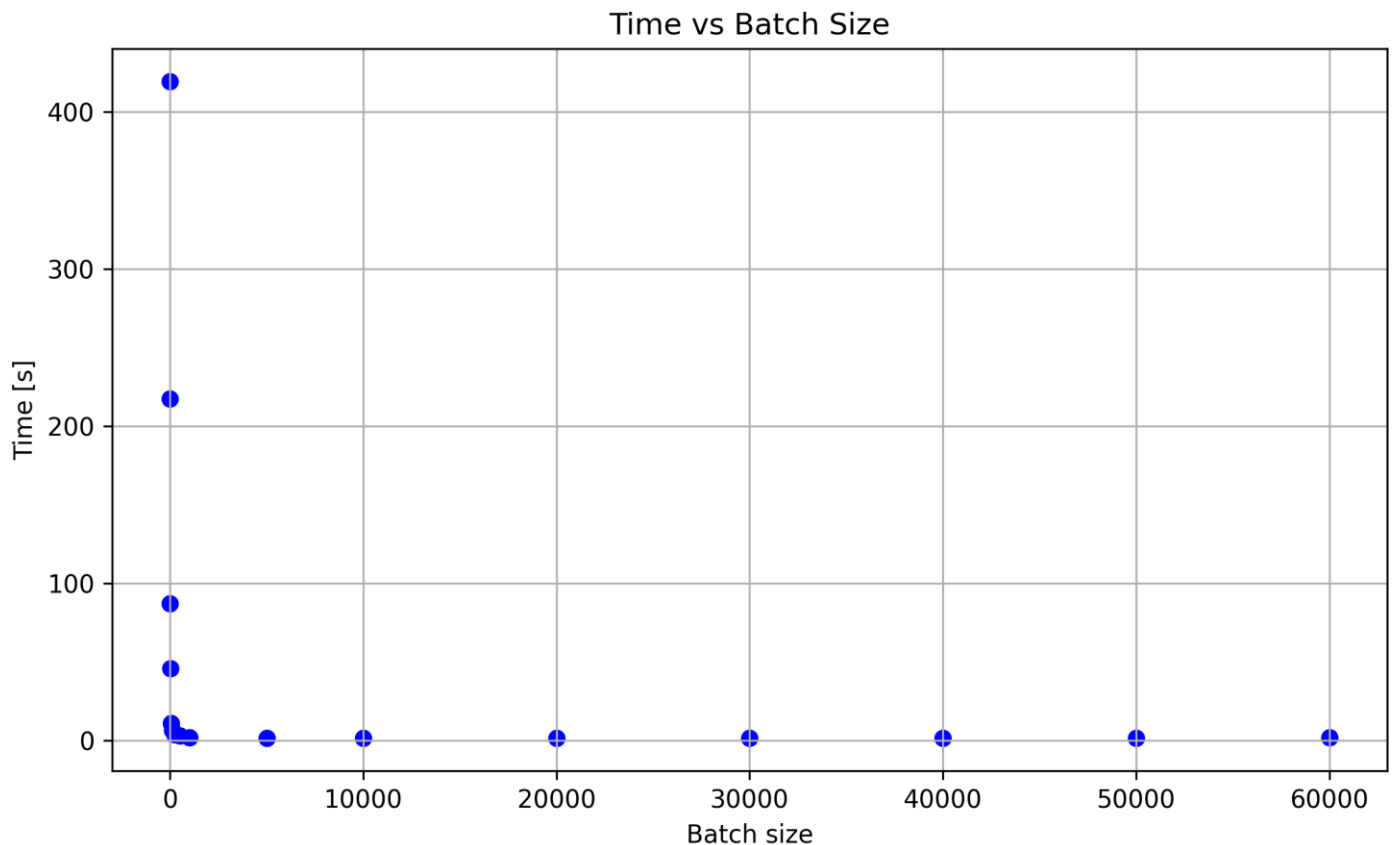
```
33     start_time = time.time()
34     model.fit(x_train, y_train, batch_size=batch_size, epochs=500) # train the model
35     end_time = time.time()
36     val_loss, val_acc = model.evaluate(x_test, y_test) # evaluate the out of sample data with model
37
38     return val_loss, val_acc, end_time-start_time
39
40 loss, accuracy, training_time = train_model(batch_size=60000)
41
42 print(f"loss: {loss}")
43 print(f"accuracy: {accuracy}")
44 print(f"time: {training_time}")
45
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1/1 ————— 0s 239ms/step - accuracy: 0.9954 - loss: 0.0233
Epoch 500/500
1/1 ————— 0s 234ms/step - accuracy: 0.9955 - loss: 0.0232
313/313 ————— 1s 2ms/step - accuracy: 0.9646 - loss: 0.1247
loss: 0.11320257931947708
accuracy: 0.9690999984741211
time: 124.7183096408844
PS C:\Users\Dell\Desktop\studia_magisterskie\II\sieci_neuronowe\ps5>



Wraz ze wzrostem wielkości partii rosta wartość funkcji straty, co potwierdza zależność odwrotną między stratą a dokładnością.



Zwiększanie wielkości partii skróciło czas trenowania modelu. Na początku zwiększanie wielkości partii znacząco skracało czas uczenia, jednak przy *batch size* ≈ 250 tempo tego spadku wyraźnie zwolniło — czas treningu zmniejszył się jedynie z 3,84 s do 2,99 s, a dalsze zwiększanie partii przynosiło coraz mniejsze korzyści. Dla wartości rzędu kilku tysięcy próbek (około 5000) czas trenowania ustabilizował się w okolicach 1,6 s, z różnicami rzędu kilkudziesięciu milisekund.

Wnioski

Wraz ze wzrostem wielkości *batch size* czas potrzebny na przetworzenie jednej epoki treningu modelu znacznie się wydłuża. Z kolei zmniejszenie wielkości partii danych skraca czas trwania pojedynczej epoki, ale zwykle wymaga większej liczby epok, ponieważ wagi sieci aktualizowane są rzadziej w trakcie jednej epoki. Dodatkowo mały *batch size* może sprzyjać przeuczeniu modelu – co potwierdzają nasze pomiary, w których przy *batch size* = 1 uzyskaliśmy niższą dokładność na zbiorze testowym niż przy wartości 2

Zad2 – Ocena wpływu parametrów sieci neuronowych na szybkość działania oraz jakość uzyskanych wyników:

Zdecydowaliśmy się przetestować następujące parametry:

- funkcje aktywacji dla warstw ukrytych: relu, leaky_relu, tanh,
- optymalizatory: Adam, AdamW, SGD,
- struktury sieci: 32x1, 32x16x1, 32x32x1, 64x64x32x1, 128x128x64x1, 256x256x128x1.

Dla każdej sieci zastosowano liniową funkcję aktywacji w warstwie wyjściowej. W rezultacie powstało 54 różnych wariantów modeli. Wszystkie zostały wytrenowane na obu zbiorach danych podanych w zadaniu, zarówno w formie surowej, jak i ustandaryzowanej. Każdy trening był powtarzany dziesięciokrotnie w celu uśrednienia wyników. Modele uczyliśmy do momentu gdy na przestrzeni ostatnich 10 epok nie było poprawy mse na zbiorze walidacyjnym, po czym braliśmy najlepsze wagi.

Pierwszą obserwacją było, że modele wykorzystujące tangens hiperboliczny jako funkcję aktywacji w warstwach ukrytych nie uczyły się na danych po standaryzacji. Próbowaliśmy rozwiązać ten problem, testując różne metody normalizacji, takie jak: standardowa standaryzacja ze średnią równą 0 i odchyleniem standardowym równym 1, normalizacja LeCun ze średnią równą 0 i odchyleniem standardowym równym 1 / 3 oraz przeskalowanie danych metodą Min-Max do zakresu $[-1, 1]$. Niestety żadne z tych rozwiązań nie przyniosło znaczącej poprawy.

```
=== StandardScaler ===
Train: min = -1.7289779994041528 max = 1.733900587941452 mean = -3.3306690738754695e-17
Val:   min = -1.6710702432789515 max = 1.7107370699463564 mean = 0.012305371851539985
Test:  min = -1.7289779994041528 max = 1.7338971250663275 mean = 0.0024595351280866317

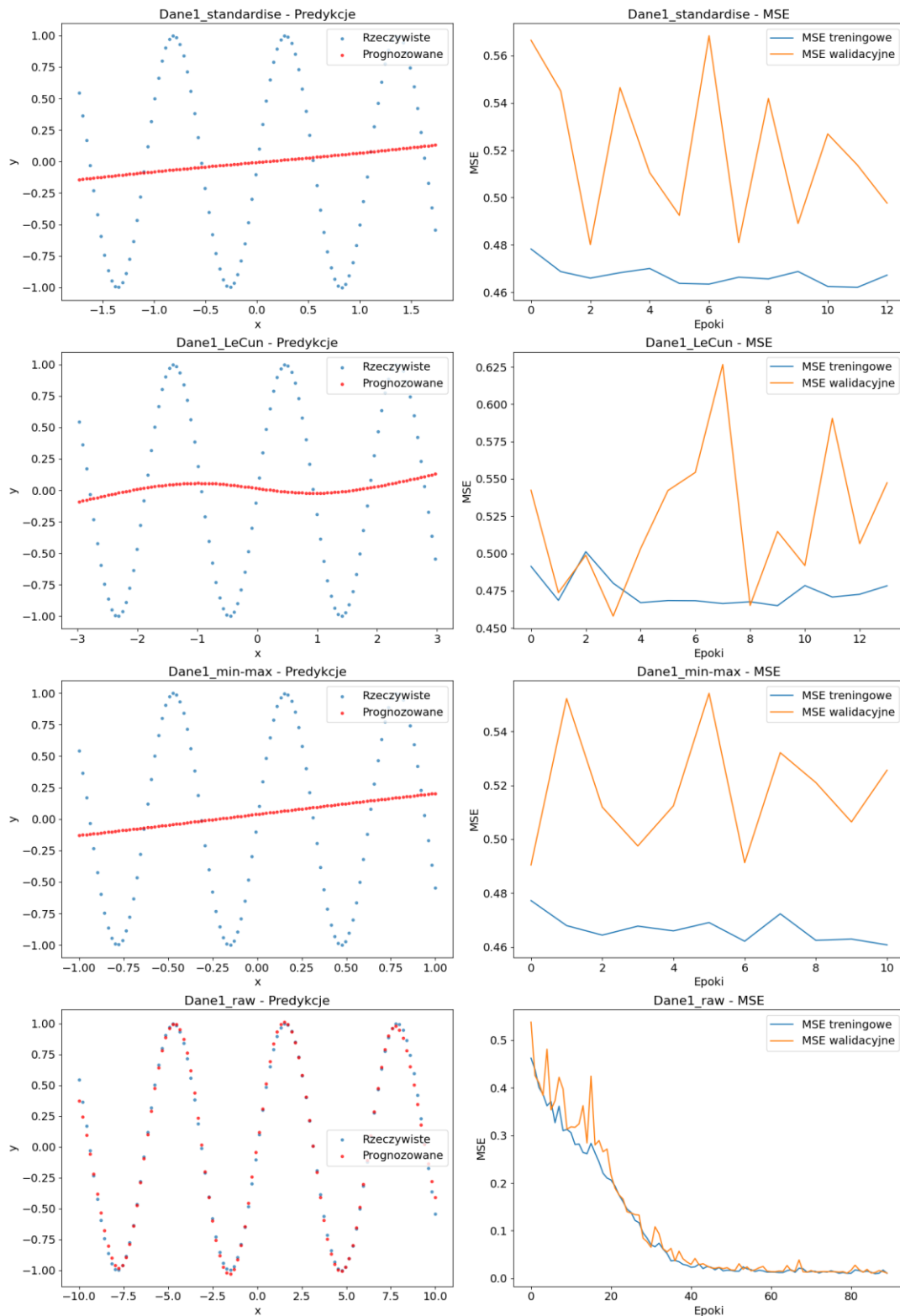
=== LeCun ===
Train: min = -2.9964956662117035 max = 3.005027015496451 mean = -5.181040781584064e-17
Val:   min = -2.8749894234744025 max = 2.900703280139739 mean = 0.0
Test:  min = -2.9719515563099903 max = 2.9719516514124424 mean = 1.7763568394002505e-17

=== MinMaxScaler ===
Train: min = -1.0 max = 1.0 mean = -0.0014215307909691535
Val:   min = -0.9665551334448667 max = 0.9866218133781867 mean = 0.005685488147845241
Test:  min = -1.0 max = 0.9999980000020001 mean = -1.015989839464423e-06

=== Raw data ===
Train: min = -10.0 max = 10.00002 mean = -0.014205322124999972
Val:   min = -9.665551 max = 9.866238 mean = 0.056864938333333344
Test:  min = -10.0 max = 10.0 mean = -1.5999999993354664e-07
```

Parametry danych wejściowych po zastosowaniu różnych metod normalizacji.

Wyniki dla: tanh, Adam, [64, 64, 32]



Wyniki uczenia na danych normalizowanych poszczególnymi metodami.

Z powodu anomalii w wynikach tanh dla danych ustandaryzowanych postanowiliśmy wykluczyć je w dalszej analizie, aby nie zaburzały i przekłamywały pozostałych wyników.

Kolejnym etapem naszej pracy była właściwa analiza otrzymanych pomiarów, którą rozpoczęliśmy od porównania funkcji aktywacji zastosowanych w warstwach ukrytych. Ocenę uzależniliśmy od zbioru danych, aby przedstawić pełniejszy obraz sytuacji oraz wykluczyć zakłócenia spowodowane pominięciem wyników dla tangensa hiperbolicznego w pierwszym zbiorze danych.

	activation	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
1	leaky_relu	Dane2	0.001942	0.003027	0.001664	0.002682	119.963889	14.642722
3	relu	Dane2	0.002667	0.004214	0.002322	0.003725	115.761111	14.979111
5	tanh	Dane2	0.007094	0.006371	0.006115	0.005854	87.080556	10.892139
4	tanh	Dane1	0.140089	0.141638	0.136370	0.141576	66.683333	10.039667
2	relu	Dane1	0.218716	0.207310	0.215787	0.207923	69.169444	11.082194
0	leaky_relu	Dane1	0.271245	0.203999	0.267275	0.202387	55.586111	8.582417

Jak można zauważyć, w drugim zbiorze danych najmniejszy błąd średniokwadratowy oraz jego odchylenie standardowe uzyskała funkcja leaky_relu, a następnie funkcja relu. Najgorzej poradził sobie tangens hiperboliczny. Co ciekawe, w pierwszym zbiorze wyniki były odwrotne – tam funkcja tanh osiągnęła najlepsze rezultaty, dodatkowo ucząc się w mniejszej liczbie epok i w krótszym czasie niż funkcja relu. W pozostałych przypadkach funkcje z niższym MSE charakteryzowały się zwykle większą średnią liczbą epok i dłuższym czasem uczenia.

	activation	data_source	preprocessing	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
2	leaky_relu	Dane2	raw	0.000784	0.001166	0.000622	0.001025	158.666667	18.985000
6	relu	Dane2	raw	0.000872	0.001349	0.000694	0.001207	137.322222	17.792778
10	tanh	Dane2	standardise	0.001465	0.000557	0.000906	0.000378	116.450000	14.269722
3	leaky_relu	Dane2	standardise	0.003099	0.003784	0.002706	0.003346	81.261111	10.300444
7	relu	Dane2	standardise	0.004462	0.005226	0.003949	0.004588	94.200000	12.165444
9	tanh	Dane2	raw	0.012722	0.004168	0.011325	0.003742	57.711111	7.514556
8	tanh	Dane1	raw	0.140089	0.141638	0.136370	0.141576	66.683333	10.039667
5	relu	Dane1	standardise	0.191070	0.208363	0.187566	0.209212	73.116667	11.546722
1	leaky_relu	Dane1	standardise	0.224243	0.205842	0.221271	0.206974	68.833333	10.364944
4	relu	Dane1	raw	0.246362	0.203083	0.244008	0.203307	65.222222	10.617667
0	leaky_relu	Dane1	raw	0.318248	0.191442	0.313279	0.187229	42.338889	6.799889

Na drugim zbiorze danych zastosowanie standaryzacji miało różnorodne efekty: dla funkcji leaky_relu i relu przyspieszyło zbieżność modeli, kosztem wzrostu błędu predykcji, natomiast dla funkcji tanh znacząco poprawiło jakość predykcji, choć czas treningu uległ podwojeniu. W przypadku pierwszego zbioru danych standaryzacja zmniejszyła błąd średniokwadratowy dla relu i leaky_relu, natomiast dla funkcji tanh uniemożliwiła skuteczne uczenie modelu. Dodatkowo standaryzacji poprawiła stabilność modeli co możemy zobaczyć po obniżeniu odchylenia standardowego.

Następnym parametrem który poddaliśmy analizie były algorytmy optymalizacji. Jak możemy zobaczyć poniżej najlepiej poradził sobie SGD (stochastic gradient descent z momentem równym 0.9) który miał zarówno najniżej mse, odchylenie standardowe oraz czas uczenia. Kolejny był Adam a po nim nieznacznie gorszy AdamW.

	optimizer	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
2	SGD	0.102583	0.169648	0.100734	0.167998	82.130303	10.599394
0	Adam	0.104288	0.175555	0.102324	0.174496	87.940909	12.630227
1	AdamW	0.104970	0.176694	0.103132	0.175875	92.239394	12.333152

	optimizer	preprocessing	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
5	SGD	standardise	0.078290	0.155669	0.076539	0.154769	93.670000	12.013533
1	Adam	standardise	0.085962	0.167235	0.084436	0.167054	80.730000	11.568133
3	AdamW	standardise	0.090352	0.172324	0.088864	0.172401	85.916667	11.606700
2	AdamW	raw	0.117152	0.179584	0.115022	0.178083	97.508333	12.938528
0	Adam	raw	0.119559	0.181014	0.117230	0.179334	93.950000	13.515306
4	SGD	raw	0.122827	0.178168	0.120897	0.175958	72.513889	9.420944

Standaryzacja danych miała istotny wpływ na skuteczność optymalizatorów. Największym beneficjentem okazał się SGD, który na danych ustandaryzowanych osiągnął najlepsze wyniki, podczas gdy na danych surowych wypadł najgorzej. Z kolei AdamW przesunął się w przeciwnym kierunku – na danych surowych był najlepszy, a na danych ustandaryzowanych zajął ostatnie miejsce.

Ostatnim badanym przez nas parametrem była ilość neuronów oraz warstw sieci neuronowej. Tutaj bez zaskoczeń wraz ze wzrostem złożoności sieci wzrastała poprawność predykcji i stabilność modelu, ale ku naszemu zaskoczeniu spadała ilość epok treningu co poskutkowało również spadkiem czasu treningu.

	structure	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
1	[256, 256, 128]	0.030739	0.087252	0.028499	0.085410	71.103030	11.176152
0	[128, 128, 64]	0.033655	0.093814	0.031300	0.092155	76.487879	11.029727
5	[64, 64, 32]	0.050529	0.118743	0.048008	0.116440	81.978788	11.683424
3	[32, 32]	0.149021	0.186327	0.147800	0.184947	83.057576	10.994424
2	[32, 16]	0.159307	0.197246	0.157985	0.195133	95.427273	12.357788
4	[32]	0.200431	0.223925	0.198787	0.223289	116.566667	13.884030

	structure	preprocessing	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
3	[256, 256, 128]	standardise	0.008255	0.017258	0.006029	0.016240	80.493333	12.298333
1	[128, 128, 64]	standardise	0.009040	0.018734	0.006788	0.018855	86.906667	12.107000
11	[64, 64, 32]	standardise	0.019578	0.056211	0.017611	0.056794	87.113333	12.317667
2	[256, 256, 128]	raw	0.049476	0.113876	0.047225	0.111412	63.277778	10.241000
0	[128, 128, 64]	raw	0.054166	0.122283	0.051727	0.119959	67.805556	10.132000
10	[64, 64, 32]	raw	0.076322	0.147694	0.073338	0.144276	77.700000	11.154889
7	[32, 32]	standardise	0.128106	0.181407	0.127875	0.181784	80.213333	10.622200
5	[32, 16]	standardise	0.158121	0.206435	0.156964	0.205331	80.466667	10.433600
4	[32, 16]	raw	0.160295	0.189824	0.158836	0.186786	107.894444	13.961278
6	[32, 32]	raw	0.166450	0.189066	0.164404	0.186418	85.427778	11.304611
9	[32]	standardise	0.186107	0.227087	0.184409	0.225422	105.440000	12.597933
8	[32]	raw	0.212367	0.221179	0.210768	0.221410	125.838889	14.955778

Standaryzacja danych poprawiła jakość predykcji modeli oraz ich stabilność kosztem czasu uczenia.

Ostatnim etapem naszej analizy było zidentyfikowanie najlepszych i najgorszych konfiguracji parametrów. W wygenerowanych tabelach zestawiliśmy po dziesięć najlepszych wyników dla każdego zbioru danych – najpierw dla pierwszego zbioru, a następnie dla drugiego

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	tanh	Adam	[128, 128, 64]	Dane1	0.009849	0.000606	0.002797	0.000562	69.90	11.3450
1	tanh	Adam	[64, 64, 32]	Dane1	0.009840	0.001541	0.002925	0.001741	97.60	15.6820
2	tanh	AdamW	[64, 64, 32]	Dane1	0.010048	0.001241	0.003083	0.001582	92.40	13.5940
3	tanh	AdamW	[128, 128, 64]	Dane1	0.010301	0.001180	0.003401	0.001527	68.60	10.5550
4	tanh	Adam	[256, 256, 128]	Dane1	0.011356	0.001120	0.004161	0.001329	64.90	11.1050
5	tanh	AdamW	[256, 256, 128]	Dane1	0.011009	0.001545	0.004545	0.002321	65.80	11.4850
6	relu	Adam	[128, 128, 64]	Dane1	0.012133	0.002150	0.006893	0.002503	110.70	19.2355
7	relu	AdamW	[256, 256, 128]	Dane1	0.014957	0.009003	0.009864	0.009466	81.75	13.8605
8	tanh	SGD	[128, 128, 64]	Dane1	0.021039	0.010708	0.016036	0.012918	57.50	8.4890
9	relu	Adam	[256, 256, 128]	Dane1	0.022831	0.026569	0.018595	0.029158	74.10	15.7270
10	leaky_relu	SGD	[64, 64, 32]	Dane2	0.000476	0.000124	0.000361	0.000121	102.90	12.3565
11	leaky_relu	SGD	[256, 256, 128]	Dane2	0.000489	0.000142	0.000367	0.000127	85.70	11.0135
12	relu	SGD	[256, 256, 128]	Dane2	0.000461	0.000068	0.000377	0.000101	96.75	12.5960
13	leaky_relu	SGD	[128, 128, 64]	Dane2	0.000512	0.000186	0.000388	0.000175	95.90	11.8300
14	relu	SGD	[128, 128, 64]	Dane2	0.000494	0.000071	0.000410	0.000091	88.65	10.9490
15	relu	SGD	[64, 64, 32]	Dane2	0.000547	0.000242	0.000461	0.000242	95.65	11.8395
16	leaky_relu	SGD	[32, 32]	Dane2	0.000599	0.000331	0.000466	0.000310	93.50	11.1205
17	leaky_relu	AdamW	[256, 256, 128]	Dane2	0.000501	0.000222	0.000477	0.000259	74.10	10.4590
18	relu	SGD	[32, 32]	Dane2	0.000599	0.000286	0.000500	0.000271	102.75	12.3175
19	leaky_relu	SGD	[32, 16]	Dane2	0.000707	0.000461	0.000557	0.000399	110.80	13.0930

Na pierwszym zbiorze danych najlepszą jakość predykcji dawało wykorzystanie tangensa hiperbolicznego z optymalizatorem Adam i nieznacznie gorszym AdamW. Dodatkowo najlepiej sprawdziły się sieci o średniej złożoności 128x128x64 i 64x64x32. Dla drugiej kolekcji danych najlepszy był algorytm SGD wraz z funkcjami leaky_relu i relu. Co ciekawe na dziewiątym i dziesiątym miejscu znalazły się modele z bardzo prostymi strukturami zawierające odpowiednio 64 i 48 neuronów.

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	relu	SGD	[32]	Dane1	0.495588	0.129121	0.494829	0.151543	19.45	3.0095
1	leaky_relu	AdamW	[32]	Dane1	0.465793	0.018498	0.464479	0.011591	16.50	2.9215
2	leaky_relu	Adam	[32]	Dane1	0.466704	0.016851	0.463473	0.010965	15.30	2.7360
3	relu	Adam	[32]	Dane1	0.464995	0.021113	0.462251	0.013821	16.90	2.9100
4	relu	AdamW	[32]	Dane1	0.461546	0.016558	0.460119	0.010456	17.00	3.2330
5	leaky_relu	SGD	[32]	Dane1	0.463938	0.047749	0.454167	0.046245	25.30	3.7560
6	leaky_relu	Adam	[32, 16]	Dane1	0.453220	0.061791	0.449677	0.058047	22.00	3.8275
7	leaky_relu	Adam	[32, 32]	Dane1	0.439472	0.059539	0.438811	0.058887	25.35	4.5120
8	leaky_relu	AdamW	[32, 16]	Dane1	0.442288	0.082422	0.438029	0.078407	29.55	4.7785
9	leaky_relu	AdamW	[32, 32]	Dane1	0.433711	0.073234	0.431912	0.072553	27.10	4.7945
10	tanh	AdamW	[32, 16]	Dane2	0.009415	0.007948	0.008216	0.007413	102.65	12.6160
11	tanh	Adam	[32, 16]	Dane2	0.008913	0.007346	0.007792	0.006884	101.20	12.3850
12	tanh	AdamW	[32, 32]	Dane2	0.008186	0.006959	0.007137	0.006441	92.80	11.6465
13	tanh	Adam	[32, 32]	Dane2	0.008038	0.006854	0.007006	0.006319	92.95	11.4645
14	tanh	Adam	[64, 64, 32]	Dane2	0.008123	0.007122	0.006930	0.006568	48.40	7.0335
15	tanh	AdamW	[128, 128, 64]	Dane2	0.007727	0.006269	0.006627	0.005771	32.55	4.9650
16	tanh	AdamW	[64, 64, 32]	Dane2	0.007692	0.007361	0.006543	0.006799	52.00	7.1540
17	tanh	SGD	[128, 128, 64]	Dane2	0.007232	0.006412	0.006216	0.005787	71.30	9.1655
18	tanh	Adam	[128, 128, 64]	Dane2	0.007265	0.006491	0.006183	0.005912	34.95	5.1800
19	tanh	Adam	[32]	Dane2	0.007064	0.005699	0.006133	0.005247	183.95	21.1870

Najgorsze na pierwszym zbiorze danych były proste sieci, a na drugim modele wykorzystujące funkcje tanh.

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	tanh	Adam	[128, 128, 64]	Dane1	0.009849	0.000606	0.002797	0.000562	69.90	11.3450
1	tanh	Adam	[256, 256, 128]	Dane1	0.011356	0.001120	0.004161	0.001329	64.90	11.1050
2	tanh	AdamW	[128, 128, 64]	Dane1	0.010301	0.001180	0.003401	0.001527	68.60	10.5550
3	tanh	AdamW	[64, 64, 32]	Dane1	0.010048	0.001241	0.003083	0.001582	92.40	13.5940
4	tanh	Adam	[64, 64, 32]	Dane1	0.009840	0.001541	0.002925	0.001741	97.60	15.6820
5	tanh	AdamW	[256, 256, 128]	Dane1	0.011009	0.001545	0.004545	0.002321	65.80	11.4850
6	relu	Adam	[128, 128, 64]	Dane1	0.012133	0.002150	0.006893	0.002503	110.70	19.2355
7	relu	AdamW	[256, 256, 128]	Dane1	0.014957	0.009003	0.009864	0.009466	81.75	13.8605
8	relu	AdamW	[32]	Dane1	0.461546	0.016558	0.460119	0.010456	17.00	3.2330
9	leaky_relu	Adam	[32]	Dane1	0.466704	0.016851	0.463473	0.010965	15.30	2.7360
10	relu	SGD	[128, 128, 64]	Dane2	0.000494	0.000071	0.000410	0.000091	88.65	10.9490
11	relu	SGD	[256, 256, 128]	Dane2	0.000461	0.000068	0.000377	0.000101	96.75	12.5960
12	leaky_relu	SGD	[64, 64, 32]	Dane2	0.000476	0.000124	0.000361	0.000121	102.90	12.3565
13	leaky_relu	SGD	[256, 256, 128]	Dane2	0.000489	0.000142	0.000367	0.000127	85.70	11.0135
14	leaky_relu	SGD	[128, 128, 64]	Dane2	0.000512	0.000186	0.000388	0.000175	95.90	11.8300
15	relu	SGD	[64, 64, 32]	Dane2	0.000547	0.000242	0.000461	0.000242	95.65	11.8395
16	leaky_relu	AdamW	[256, 256, 128]	Dane2	0.000501	0.000222	0.000477	0.000259	74.10	10.4590
17	relu	SGD	[32, 32]	Dane2	0.000599	0.000286	0.000500	0.000271	102.75	12.3175
18	leaky_relu	SGD	[32, 32]	Dane2	0.000599	0.000331	0.000466	0.000310	93.50	11.1205
19	relu	SGD	[32, 16]	Dane2	0.000657	0.000301	0.000582	0.000343	134.75	15.9970

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	tanh	SGD	[256, 256, 128]	Dane1	0.307299	0.237120	0.296824	0.230588	30.80	5.2680
1	leaky_relu	AdamW	[64, 64, 32]	Dane1	0.171995	0.205499	0.167089	0.202072	99.45	14.3920
2	leaky_relu	Adam	[64, 64, 32]	Dane1	0.170883	0.200792	0.165849	0.194223	95.05	13.8005
3	leaky_relu	Adam	[128, 128, 64]	Dane1	0.119973	0.187868	0.114702	0.184580	84.50	12.5010
4	leaky_relu	SGD	[64, 64, 32]	Dane1	0.188565	0.177446	0.183250	0.175469	61.30	8.6725
5	relu	SGD	[32, 32]	Dane1	0.275252	0.172022	0.272136	0.170984	56.65	8.0140
6	leaky_relu	AdamW	[128, 128, 64]	Dane1	0.101772	0.175270	0.096387	0.170880	88.90	13.2700
7	relu	AdamW	[64, 64, 32]	Dane1	0.084206	0.168912	0.078880	0.167272	129.85	18.5415
8	relu	Adam	[32, 32]	Dane1	0.321982	0.169792	0.320228	0.165507	80.00	12.3930
9	relu	SGD	[128, 128, 64]	Dane1	0.126874	0.167025	0.123586	0.165477	68.70	9.9285
10	tanh	AdamW	[32, 16]	Dane2	0.009415	0.007948	0.008216	0.007413	102.65	12.6160
11	tanh	Adam	[32, 16]	Dane2	0.008913	0.007346	0.007792	0.006884	101.20	12.3850
12	tanh	AdamW	[64, 64, 32]	Dane2	0.007692	0.007361	0.006543	0.006799	52.00	7.1540
13	tanh	Adam	[64, 64, 32]	Dane2	0.008123	0.007122	0.006930	0.006568	48.40	7.0335
14	tanh	AdamW	[32, 32]	Dane2	0.008186	0.006959	0.007137	0.006441	92.80	11.6465
15	tanh	Adam	[32, 32]	Dane2	0.008038	0.006854	0.007006	0.006319	92.95	11.4645
16	tanh	SGD	[64, 64, 32]	Dane2	0.006290	0.006902	0.005380	0.006174	79.75	9.8875
17	relu	AdamW	[128, 128, 64]	Dane2	0.004521	0.006578	0.004100	0.005973	65.85	9.0645
18	tanh	Adam	[128, 128, 64]	Dane2	0.007265	0.006491	0.006183	0.005912	34.95	5.1800
19	tanh	SGD	[128, 128, 64]	Dane2	0.007232	0.006412	0.006216	0.005787	71.30	9.1655

W większości przypadków modele były stabilne dla tych samych przypadków dla których mediana błędów średniokwadratowych była najniższa.

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	relu	AdamW	[64, 64, 32]	Dane1	0.084206	0.168912	0.078880	0.167272	129.85	18.5415
1	relu	Adam	[64, 64, 32]	Dane1	0.054998	0.110671	0.050560	0.109725	121.35	21.8030
2	relu	Adam	[128, 128, 64]	Dane1	0.012133	0.002150	0.006893	0.002503	110.70	19.2355
3	leaky_relu	AdamW	[64, 64, 32]	Dane1	0.171995	0.205499	0.167089	0.202072	99.45	14.3920
4	relu	AdamW	[128, 128, 64]	Dane1	0.024412	0.028007	0.020592	0.030518	99.45	14.7790
5	tanh	Adam	[64, 64, 32]	Dane1	0.009840	0.001541	0.002925	0.001741	97.60	15.6820
6	leaky_relu	Adam	[64, 64, 32]	Dane1	0.170883	0.200792	0.165849	0.194223	95.05	13.8005
7	tanh	AdamW	[64, 64, 32]	Dane1	0.010048	0.001241	0.003083	0.001582	92.40	13.5940
8	leaky_relu	AdamW	[128, 128, 64]	Dane1	0.101772	0.175270	0.096387	0.170880	88.90	13.2700
9	tanh	AdamW	[32, 16]	Dane1	0.226690	0.056427	0.227534	0.052435	85.90	12.0470
10	leaky_relu	AdamW	[32]	Dane2	0.002092	0.001902	0.001736	0.001733	286.05	32.9170
11	relu	Adam	[32]	Dane2	0.003070	0.002828	0.002560	0.002453	237.25	26.8775
12	relu	AdamW	[32]	Dane2	0.003584	0.002973	0.002897	0.002463	236.10	26.9135
13	leaky_relu	Adam	[32]	Dane2	0.003144	0.002370	0.002654	0.002130	193.20	22.4410
14	tanh	AdamW	[32]	Dane2	0.007044	0.005940	0.006065	0.005377	192.95	22.1950
15	tanh	Adam	[32]	Dane2	0.007064	0.005699	0.006133	0.005247	183.95	21.1870
16	tanh	SGD	[32, 16]	Dane2	0.001275	0.002629	0.000984	0.002328	170.55	20.1895
17	relu	AdamW	[32, 16]	Dane2	0.002847	0.003396	0.002499	0.003248	170.15	20.1815
18	leaky_relu	SGD	[32]	Dane2	0.001181	0.000609	0.001048	0.000604	156.70	18.0290
19	relu	SGD	[32]	Dane2	0.001129	0.001118	0.000925	0.000890	149.50	17.3065

Najwięcej epok uczenia potrzebowały optymalizatory Adam i AdmaW. Dodatkowo dla pierwszego zbioru danych najdłużej trenowały się sieci o średnio złożonej strukturze, najprawdopodobniej dlatego, że prostsze sieci nie były w stanie nauczyć się wzorca z danych. Na drugim zbiorze danych najwięcej epok potrzebowały sieci z jedną 32 neuronową warstwą ukrytą.

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	leaky_relu	Adam	[32]	Dane1	0.466704	0.016851	0.463473	0.010965	15.30	2.7360
1	leaky_relu	AdamW	[32]	Dane1	0.465793	0.018498	0.464479	0.011591	16.50	2.9215
2	relu	Adam	[32]	Dane1	0.464995	0.021113	0.462251	0.013821	16.90	2.9100
3	relu	AdamW	[32]	Dane1	0.461546	0.016558	0.460119	0.010456	17.00	3.2330
4	relu	SGD	[32]	Dane1	0.495588	0.129121	0.494829	0.151543	19.45	3.0095
5	leaky_relu	Adam	[32, 16]	Dane1	0.453220	0.061791	0.449677	0.058047	22.00	3.8275
6	leaky_relu	SGD	[32]	Dane1	0.463938	0.047749	0.454167	0.046245	25.30	3.7560
7	leaky_relu	Adam	[32, 32]	Dane1	0.439472	0.059539	0.438811	0.058887	25.35	4.5120
8	leaky_relu	AdamW	[32, 32]	Dane1	0.433711	0.073234	0.431912	0.072553	27.10	4.7945
9	leaky_relu	AdamW	[32, 16]	Dane1	0.442288	0.082422	0.438029	0.078407	29.55	4.7785
10	tanh	AdamW	[128, 128, 64]	Dane2	0.007727	0.006269	0.006627	0.005771	32.55	4.9650
11	tanh	Adam	[256, 256, 128]	Dane2	0.006958	0.005215	0.005992	0.004910	34.10	5.5285
12	tanh	AdamW	[256, 256, 128]	Dane2	0.006866	0.005372	0.005947	0.004997	34.80	5.5535
13	tanh	Adam	[128, 128, 64]	Dane2	0.007265	0.006491	0.006183	0.005912	34.95	5.1800
14	tanh	Adam	[64, 64, 32]	Dane2	0.008123	0.007122	0.006930	0.006568	48.40	7.0335
15	tanh	AdamW	[64, 64, 32]	Dane2	0.007692	0.007361	0.006543	0.006799	52.00	7.1540
16	leaky_relu	AdamW	[64, 64, 32]	Dane2	0.004454	0.004725	0.004038	0.004359	65.15	8.6460
17	relu	AdamW	[128, 128, 64]	Dane2	0.004521	0.006578	0.004100	0.005973	65.85	9.0645
18	relu	Adam	[128, 128, 64]	Dane2	0.004051	0.006621	0.003487	0.005699	66.40	11.5930
19	relu	AdamW	[64, 64, 32]	Dane2	0.005713	0.006271	0.005110	0.005669	67.20	9.0185

Dla pierwszego zbioru danych najkrócej uczyły się proste sieci neuronowe, ponieważ nie będąc w stanie nauczyć się wzorca praktycznie natychmiast kończyły trening. Na drugiej kolekcji danych najmniej epok potrzebowały modele ze złożoną strukturą i funkcją aktywacji tanh, gdyż bardzo szybko wyłapywały wzorzec z danych i uzyskiwały satysfakcjonujący błąd średniokwadratowy.

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	relu	Adam	[64, 64, 32]	Dane1	0.054998	0.110671	0.050560	0.109725	121.35	21.8030
1	relu	Adam	[128, 128, 64]	Dane1	0.012133	0.002150	0.006893	0.002503	110.70	19.2355
2	relu	AdamW	[64, 64, 32]	Dane1	0.084206	0.168912	0.078880	0.167272	129.85	18.5415
3	relu	Adam	[256, 256, 128]	Dane1	0.022831	0.026569	0.018595	0.029158	74.10	15.7270
4	tanh	Adam	[64, 64, 32]	Dane1	0.009840	0.001541	0.002925	0.001741	97.60	15.6820
5	relu	AdamW	[128, 128, 64]	Dane1	0.024412	0.028007	0.020592	0.030518	99.45	14.7790
6	leaky_relu	AdamW	[256, 256, 128]	Dane1	0.028722	0.025347	0.025356	0.027699	81.15	14.4820
7	leaky_relu	AdamW	[64, 64, 32]	Dane1	0.171995	0.205499	0.167089	0.202072	99.45	14.3920
8	relu	AdamW	[256, 256, 128]	Dane1	0.014957	0.009003	0.009864	0.009466	81.75	13.8605
9	leaky_relu	Adam	[64, 64, 32]	Dane1	0.170883	0.200792	0.165849	0.194223	95.05	13.8005
10	leaky_relu	AdamW	[32]	Dane2	0.002092	0.001902	0.001736	0.001733	286.05	32.9170
11	relu	AdamW	[32]	Dane2	0.003584	0.002973	0.002897	0.002463	236.10	26.9135
12	relu	Adam	[32]	Dane2	0.003070	0.002828	0.002560	0.002453	237.25	26.8775
13	leaky_relu	Adam	[32]	Dane2	0.003144	0.002370	0.002654	0.002130	193.20	22.4410
14	tanh	AdamW	[32]	Dane2	0.007044	0.005940	0.006065	0.005377	192.95	22.1950
15	tanh	Adam	[32]	Dane2	0.007064	0.005699	0.006133	0.005247	183.95	21.1870
16	tanh	SGD	[32, 16]	Dane2	0.001275	0.002629	0.000984	0.002328	170.55	20.1895
17	relu	AdamW	[32, 16]	Dane2	0.002847	0.003396	0.002499	0.003248	170.15	20.1815
18	relu	Adam	[32, 16]	Dane2	0.004148	0.004237	0.003391	0.003594	120.40	18.6965
19	leaky_relu	SGD	[32]	Dane2	0.001181	0.000609	0.001048	0.000604	156.70	18.0290

	activation	optimizer	structure	data_source	mse_train_mean	mse_train_std	mse_test_mean	mse_test_std	epochs_run_mean	fit_time_sec_mean
0	leaky_relu	Adam	[32]	Dane1	0.466704	0.016851	0.463473	0.010965	15.30	2.7360
1	relu	Adam	[32]	Dane1	0.464995	0.021113	0.462251	0.013821	16.90	2.9100
2	leaky_relu	AdamW	[32]	Dane1	0.465793	0.018498	0.464479	0.011591	16.50	2.9215
3	relu	SGD	[32]	Dane1	0.495588	0.129121	0.494829	0.151543	19.45	3.0095
4	relu	AdamW	[32]	Dane1	0.461546	0.016558	0.460119	0.010456	17.00	3.2330
5	leaky_relu	SGD	[32]	Dane1	0.463938	0.047749	0.454167	0.046245	25.30	3.7560
6	leaky_relu	Adam	[32, 16]	Dane1	0.453220	0.061791	0.449677	0.058047	22.00	3.8275
7	leaky_relu	Adam	[32, 32]	Dane1	0.439472	0.059539	0.438811	0.058887	25.35	4.5120
8	leaky_relu	AdamW	[32, 16]	Dane1	0.442288	0.082422	0.438029	0.078407	29.55	4.7785
9	leaky_relu	AdamW	[32, 32]	Dane1	0.433711	0.073234	0.431912	0.072553	27.10	4.7945
10	tanh	AdamW	[128, 128, 64]	Dane2	0.007727	0.006269	0.006627	0.005771	32.55	4.9650
11	tanh	Adam	[128, 128, 64]	Dane2	0.007265	0.006491	0.006183	0.005912	34.95	5.1800
12	tanh	Adam	[256, 256, 128]	Dane2	0.006958	0.005215	0.005992	0.004910	34.10	5.5285
13	tanh	AdamW	[256, 256, 128]	Dane2	0.006866	0.005372	0.005947	0.004997	34.80	5.5535
14	tanh	Adam	[64, 64, 32]	Dane2	0.008123	0.007122	0.006930	0.006568	48.40	7.0335
15	tanh	AdamW	[64, 64, 32]	Dane2	0.007692	0.007361	0.006543	0.006799	52.00	7.1540
16	leaky_relu	AdamW	[64, 64, 32]	Dane2	0.004454	0.004725	0.004038	0.004359	65.15	8.6460
17	relu	AdamW	[64, 64, 32]	Dane2	0.005713	0.006271	0.005110	0.005669	67.20	9.0185
18	relu	AdamW	[128, 128, 64]	Dane2	0.004521	0.006578	0.004100	0.005973	65.85	9.0645
19	tanh	SGD	[128, 128, 64]	Dane2	0.007232	0.006412	0.006216	0.005787	71.30	9.1655

Czas uczenia modelu w głównej mierze zależał od ilości epok.

Wnioski

Najważniejszym wnioskiem do jakiego doszliśmy w czasie naszych eksperymentów jest to, że dobór właściwych parametrów uczenia w dużej mierze zależy od danych na których model będzie trenowany. Dobrze widać to w przypadku tangensa hiperbolicznego który świetnie sobie poradził z surowymi danymi ze zbioru pierwszego, ale gdy znormalizowały ten zbiór danych przestał działać, a w przypadku drugiej kolekcji przegrał z relu i leaky_relu.

Kolejnym przykładem może być algorytm SGD który świetnie wypada przy danych ustandaryzowanych, ale na surowych przegrywa z konkurentami.

Następnym wnioskiem do jakiego doszliśmy jest fakt, że więcej neuronów nie znaczy lepiej i czasami prostsze sieci mogą uzyskiwać lepsze wyniki, ale również czasami zbyt prosta struktura może prowadzić do bardzo długiego procesu uczenia, gdzie większa sieć może w ułamku tego czasu znaleźć prawidłowy wzorzec w danych, nie wspominając już o tym że zbyt mała sieć może po prostu się nie nauczyć.

W trakcie eksperymentu zauważyliśmy również że mechanizm *early stoppingu* daje wyśmienite efekty czego oznaką jest fakt że mimo stosunkowo złożonych struktur sieci w niektórych przypadkach nie mieliśmy problemu z przeuczeniem modelu.